

# Personal Expense Tracker

Artificial Intelligence – Major Project

Submitted by: Irine Micheal

Language: Python 3 | Libraries: csv, datetime, matplotlib

## 1. Project Overview

This project is a command-line Personal Expense Tracker written in Python. It lets a user record daily expenses, organise them by category, generate spending reports, view charts, and keep their data between sessions by saving it to a CSV file. All user input is validated with try-except blocks so the program does not crash on invalid entries.

## 2. Features

- **Add an expense** – amount, category (chosen from a numbered list) and date (YYYY-MM-DD, or today by default).
- **View all expenses** – displayed as a formatted table.
- **Generate report** – total spent, average expense, highest expense, spending by category and total spending by month.
- **Charts (Matplotlib)** – bar and pie chart of spending by category, and a bar chart of total spending by month.
- **Delete an expense** – choose an expense by number; entering 0 cancels.
- **Monthly budget warning** – the user sets a budget, which is saved to a file. After each new expense the program warns at 80% of the budget and again when it is exceeded.
- **File handling** – expenses are saved to `expenses.csv` and loaded on start-up; the budget is stored in `budget.txt`.
- **Error handling** – invalid numbers, invalid dates, out-of-range choices, a missing file and damaged CSV rows are all handled.

## 3. Concepts Used

| Concept                | Where it is used                                                                                                                        |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Lists and dictionaries | Each expense is a dictionary (amount, category, date) stored in a list.                                                                 |
| Functions              | <code>add_expense()</code> , <code>view_expenses()</code> , <code>generate_report()</code> , <code>save_expenses()</code> , and others. |
| Control flow           | <code>if / elif / else</code> for the menu; <code>while</code> loop to keep the program running; <code>for</code> loops for reports.    |
| File handling          | <code>csv.DictWriter</code> and <code>csv.DictReader</code> for expenses; plain text file for the budget.                               |

|                    |                                                           |
|--------------------|-----------------------------------------------------------|
| Exception handling | try-except for ValueError, FileNotFoundError and OSError. |
| Data visualisation | Matplotlib bar and pie charts.                            |

## 4. How to Run

- Install Python 3 and Matplotlib: `python -m pip install matplotlib`
- Save the code below as `expense_tracker.py`.
- Run: `python expense_tracker.py`
- Use the menu (options 1 to 8). Choose option 8 to save and exit.

## Menu

1. Add an Expense
2. View All Expenses
3. Generate Report
4. Show Category Chart
5. Show Monthly Chart
6. Delete an Expense
7. Set Monthly Budget
8. Save and Exit

## 5. Source Code (expense\_tracker.py)

```
import csv

FILENAME = "expenses.csv"
BUDGET_FILE = "budget.txt"
from datetime import datetime
import matplotlib.pyplot as plt
CATEGORIES = ["Food", "Transport", "Entertainment", "Bills", "Shopping", "Other"]
monthly_budget = 0 # 0 means no budget has been set
expenses = [] # this list will hold all our expenses
def get_amount():
    while True:
        try:
            amount = float(input("Enter amount: "))
            if amount <= 0:
                print("Amount must be greater than zero.")
                continue
            return amount
        except ValueError:
            print("Invalid input. Please enter a number (e.g., 50 or 12.5).")
def get_category():
    print("Categories:")
    for i, cat in enumerate(CATEGORIES, start=1):
        print(f" {i}. {cat}")
    while True:
        try:
            choice = int(input("Choose a category number: "))
            if 1 <= choice <= len(CATEGORIES):
                return CATEGORIES[choice - 1]
            print(f"Please choose a number between 1 and {len(CATEGORIES)}.")
        except ValueError:
            print("Invalid input. Please enter a number.")

def get_date():
    while True:
        text = input("Enter date (YYYY-MM-DD) or press Enter for today: ").strip()
        if text == "":
            return datetime.now().strftime("%Y-%m-%d")
        try:
            datetime.strptime(text, "%Y-%m-%d")
            return text
        except ValueError:
            print("Invalid date. Use the format YYYY-MM-DD (e.g., 2024-10-13).")
def add_expense():
    amount = get_amount()
    category = get_category()
    date = get_date()

    expense = {"amount": amount, "category": category, "date": date}
    expenses.append(expense)
    print("Expense added successfully!")
    check_budget(date)

def set_budget():
    global monthly_budget
    while True:
        try:
            amount = float(input("Enter your monthly budget: "))
            if amount <= 0:
                print("Budget must be greater than zero.")
                continue
            monthly_budget = amount
            save_budget()
            print(f"Monthly budget set to ${amount:.2f}")

            return
        except ValueError:
            print("Invalid input. Please enter a number.")
def save_budget():
    try:
```

```

        with open(BUDGET_FILE, "w") as file:
            file.write(str(monthly_budget))
    except OSError as error:
        print(f"Could not save the budget: {error}")

def load_budget():
    global monthly_budget
    try:
        with open(BUDGET_FILE, "r") as file:
            monthly_budget = float(file.read().strip())
            print(f"Monthly budget: ${monthly_budget:.2f}")
    except (FileNotFoundError, ValueError):
        monthly_budget = 0

def check_budget(date):
    if monthly_budget == 0:
        return

    month = date[:7]
    month_total = 0
    for e in expenses:
        if e["date"][:7] == month:
            month_total += e["amount"]

    if month_total > monthly_budget:
        over = month_total - monthly_budget
        print(f"WARNING: You are over your budget for {month} by ${over:.2f}!")
    elif month_total >= 0.8 * monthly_budget:
        print(f"Careful: you have used ${month_total:.2f} of your ${monthly_budget:.2f} budget for {month}.!!")

def view_expenses():
    if not expenses:
        print("No expenses recorded yet.")
        return

    print(f"\n{'Date':<12}{'Category':<16}{'Amount':>10}")
    print("-" * 38)
    for e in expenses:
        print(f"{e['date']:<12}{e['category']:<16}${e['amount']:>9.2f}")

def delete_expense():
    if not expenses:
        print("No expenses to delete.")
        return

    print(f"\n{'No.':<5}{'Date':<12}{'Category':<16}{'Amount':>10}")
    print("-" * 43)
    for i, e in enumerate(expenses, start=1):
        print(f"{i:<5}{e['date']:<12}{e['category']:<16}${e['amount']:>9.2f}")

    try:
        number = int(input("Enter the number to delete: "))
    except ValueError:
        print("Invalid input. Please enter a number.")
        return

    if number == 0:
        print("Cancelled.")
    elif 1 <= number <= len(expenses):
        removed = expenses.pop(number - 1)
        print(f"Deleted: {removed['date']} {removed['category']} ${removed['amount']:.2f}")
    else:
        print("That number is not in the list.")

def generate_report():
    if not expenses:
        print("No expenses to report yet.")
        return

    total = 0
    highest = expenses[0]
    by_category = {}
    by_month = {}
    for e in expenses:

```

```

    total += e["amount"]

    if e["amount"] > highest["amount"]:
        highest = e

    cat = e["category"]
    by_category[cat] = by_category.get(cat, 0) + e["amount"]

    month = e["date"][:7] # Extract YYYY-MM
    by_month[month] = by_month.get(month, 0) + e["amount"]

average = total / len(expenses)

print("\n--- Expense Report ---")
print(f"Total spent:      ${total:.2f}")
print(f"Average expense: ${average:.2f}")
print(f"Highest expense: ${highest['amount']:.2f} ({highest['category']}, {highest['date']})")

print("\nSpending by category:")
for cat, amount in by_category.items():
    print(f"  {cat:<15}${amount:>9.2f}")

print("\nSpending by month:")
for month, amount in sorted(by_month.items()):
    print(f"  {month:<15}${amount:>9.2f}")
def show_chart():
    if not expenses:
        print("No expenses to chart yet.")
        return

    by_category = {}
    for e in expenses:
        cat = e["category"]
        by_category[cat] = by_category.get(cat, 0) + e["amount"]

    categories = list(by_category.keys())
    amounts = list(by_category.values())

    plt.figure(figsize=(10, 4))

    plt.subplot(1, 2, 1)
    plt.bar(categories, amounts)
    plt.title("Spending by Category")
    plt.ylabel("Amount ($)")

    plt.subplot(1, 2, 2)
    plt.pie(amounts, labels=categories, autopct="%1.1f%%")
    plt.title("Share of Total")

    plt.tight_layout()
    plt.show()
def show_monthly_chart():
    if not expenses:
        print("No expenses to chart yet.")
        return

    by_month = {}
    for e in expenses:
        month = e["date"][:7]
        by_month[month] = by_month.get(month, 0) + e["amount"]

    months = sorted(by_month.keys())
    totals = [by_month[m] for m in months]

    plt.bar(months, totals)
    plt.title("Total Spending by Month")
    plt.xlabel("Month")
    plt.ylabel("Amount ($)")
    plt.show()
def load_expenses():
    try:
        with open(FILENAME, "r", newline="") as file:
            reader = csv.DictReader(file)

```

```

        for row in reader:
            try:
                expenses.append({
                    "amount": float(row["amount"]),
                    "category": row["category"],
                    "date": row["date"],
                })
            except (ValueError, KeyError):
                print(f"Skipped a damaged row: {row}")
            print(f"Loaded {len(expenses)} expense(s) from {FILENAME}.")
    except FileNotFoundError:
        print("No saved file found. Starting with an empty list.")
def save_expenses():
    try:
        with open(FILENAME, "w", newline="") as file:
            writer = csv.DictWriter(file, fieldnames=["date", "category", "amount"])
            writer.writeheader()
            for e in expenses:
                writer.writerow(e)
            print(f"Saved {len(expenses)} expense(s) to {FILENAME}.")
    except OSError as error:
        print(f"Could not save the file: {error}")
print("Welcome to Personal Expense Tracker!")
load_expenses()
load_budget()
while True:
    print("\n1. Add an Expense")
    print("2. View All Expenses")
    print("3. Generate Report")
    print("4. Show Category Chart")
    print("5. Show Monthly Chart")
    print("6. Delete an Expense")
    print("7. Set Monthly Budget")
    print("8. Save and Exit")
    choice = input("Enter your choice: ")

    if choice == "1":
        add_expense()
    elif choice == "2":
        view_expenses()
    elif choice == "3":
        generate_report()
    elif choice == "4":
        show_chart()
    elif choice == "5":
        show_monthly_chart()
    elif choice == "6":
        delete_expense()
    elif choice == "7":
        set_budget()
    elif choice == "8":
        save_expenses()
        print("Goodbye, Have a nice day!")
        break
    else:
        print("Invalid choice. Please enter a number from 1 to 8.")

```